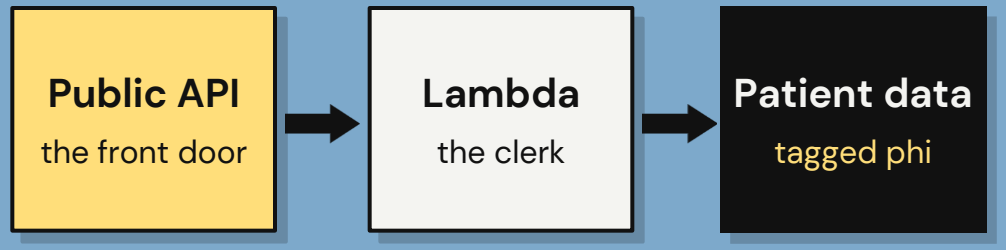
CausalFence

AWS Architecture Security Testing Dashboard

Agile Development & DevOps

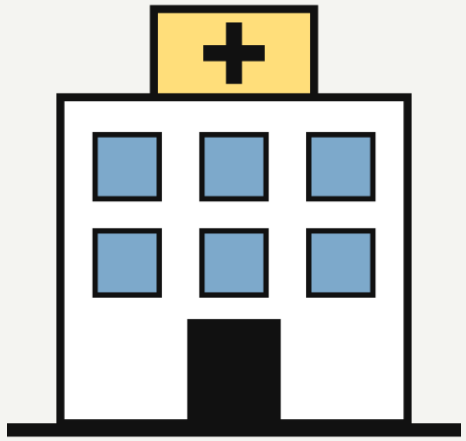


Presented by
Ruksana S
24MIS0293

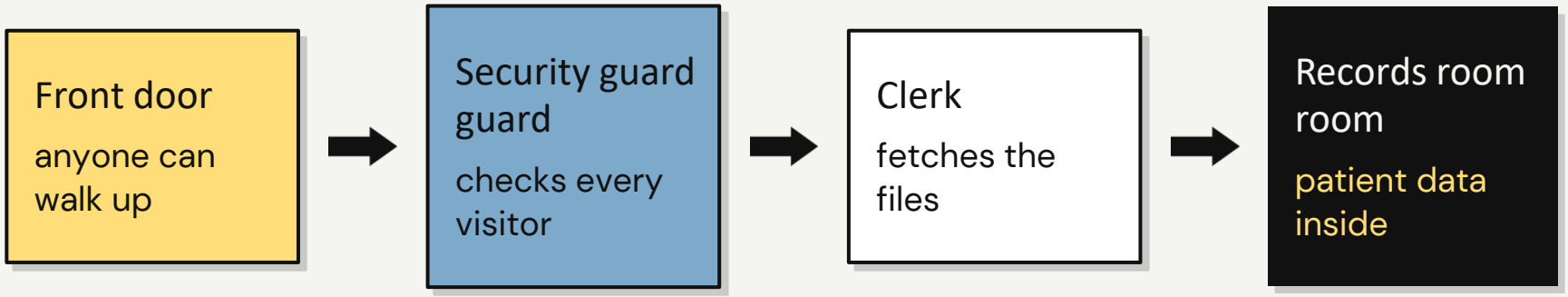
Institution
VIT
Course project

Problem Statement

RULE: everyone must pass the security guard before entering the records room.

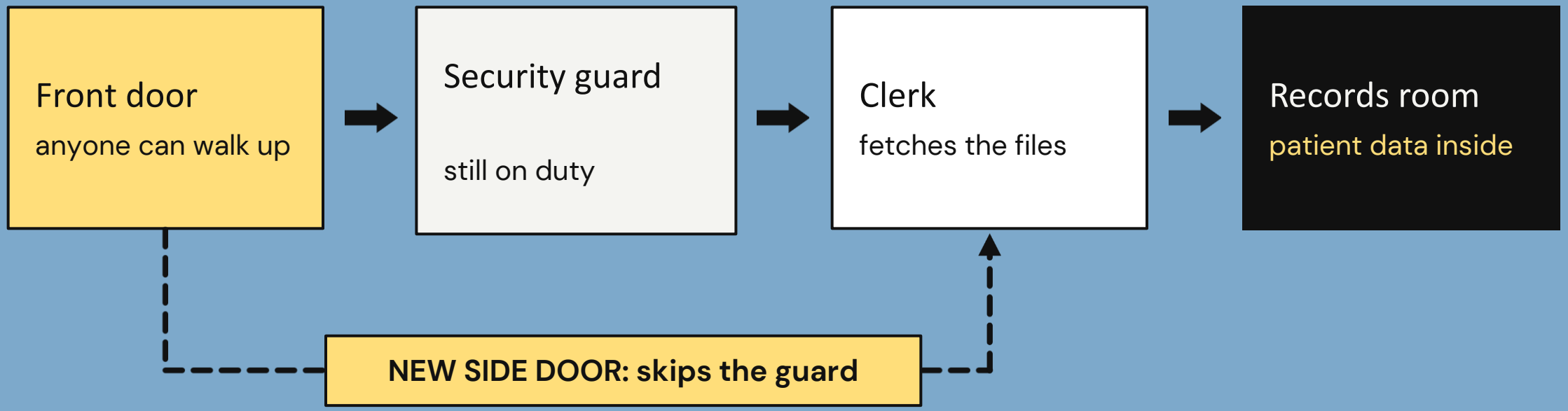


The hospital



In the cloud the front door is a public API, the guard is an authentication check, and the records room is a database of patient data.

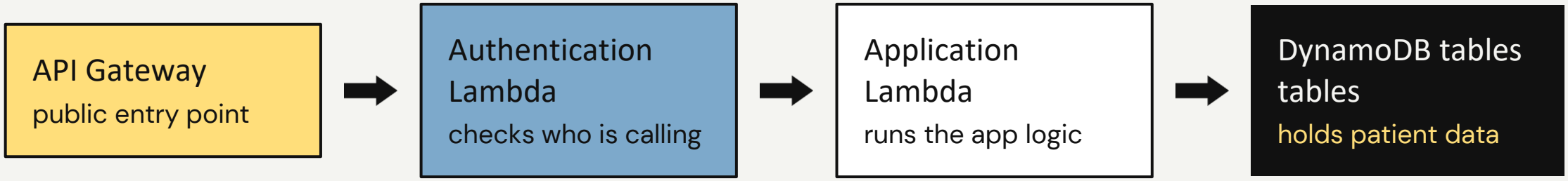
Worst Case



A stranger just walked in.

The guard is still there. The lock still works. Every part looks fine.

AWS serverless apps work the same way



NORMAL SECURITY CHECKS

Checks each part separately.

- ✓ API Gateway: auth enabled?
- ✓ Lambda: DynamoDB access?
- ✓ IAM: permission allowed?
- ✓ DynamoDB: table encrypted?

WHAT THEY MISS

No check sees the whole route.

- ⚠ DynamoDB never checks if the caller is authenticated
- ⚠ Lambda does not know which route reached it
- ⚠ IAM trusts the Lambda, not who called it
- ⚠ A new route can skip the authorizer

WHAT CAUSALFENCE DOES

A program that draws a map of the building and checks every route from the front door to the records room. If any route skips the guard, it raises an alarm.

rules it checks

AUTH_BYPASS	**HIGH**	A public entry reaches sensitive data without passing an authentication node.
LAMBDA_URL_OPEN	**HIGH**	A Lambda Function URL with AuthType NONE reaches sensitive data An open URL with no login.
OVERBROAD_IAM	**MEDIUM**	A wildcard permission such as dynamodb:* on a path from the internet.(* means every action: read , write , delete table)

```yaml
Action: "dynamodb:*"
Resource: !GetAtt ReportsTable.Arn
```

The four AWS stacks we deployed

causalfence-safe

Authorizer on every every route

Hospital template, AuthMode=safe

SAFE

causalfence-broken

Authorizer removed removed

Same template, AuthMode=broken

HIGH: AUTH_BYPASS

causalfence-urlopen

Open Lambda URL

Function URL with AuthType NONE

HIGH: LAMBDA_URL_OPEN

causalfence-wildcard

Wildcard permission

dynamodb:* on the patient table

MEDIUM: OVERBROAD_IAM

Each stack is a toy hospital: a public API, a Lambda and a DynamoDB table tagged sensitivity=phi. All four run in AWS region ap-south-1 (Mumbai).

How we created them

1. Write a SAM template: API, Lambda, and a DynamoDB table tagged sensitivity=phi.

2.
```
sam build --template-file urlopen-app.yaml
sam deploy --stack-name causalfence-urlopen
--region ap-south-1
--capabilities CAPABILITY_IAM
--resolve-s3 --no-confirm-changeset
```

3. CloudFormation creates the resources and the stack appears in the console.

Safe and broken come from one template. A parameter, AuthMode, decides whether the authorizer is attached.

```
Build Succeeded

Built Artifacts  : .aws-sam\build
Built Template   : .aws-sam\build\template.yaml

Commands you can use next
=========================
[*] Validate SAM template: sam validate
[*] Invoke Function: sam local invoke
[*] Test Function in the Cloud: sam sync --stack-name {{stack-name}} --watch
[*] Deploy: sam deploy --guided
PS C:\Users\ruksa\causalfence\infra\demo-stacks> sam deploy --stack-name cau
salfence-urlopen --region ap-south-1 --capabilities CAPABILITY_IAM --resolve
-s3 --no-confirm-changeset

        Managed S3 bucket: aws-sam-cli-managed-default-samclisourcebucket-jw
lvj4llfiof
        Auto resolution of buckets can be turned off by setting resolve_s3=F
alse
        To use a specific S3 bucket, set --s3-bucket=<bucket_name>
        Above settings can be stored in samconfig.toml
        Uploading to 343d9b09050e67bfd314d7b0d677982c  357 / 357  (100.00%)

        Deploying with following values
        ===============================
        Stack name                   : causalfence-urlopen
        Region                       : ap-south-1
        Confirm changeset            : False
        Disable rollback             : False
        Deployment s3 bucket         : aws-sam-cli-managed-default-samclisou
rcebucket-jwlvj4llfiof
        Capabilities                 : ["CAPABILITY_IAM"]
        Parameter overrides          : {}
        Signing Profiles             : {}

Initiating deployment
```

```
Waiting for changeset to be created..

CloudFormation stack changeset
-----------------------------------------------------------------------------
---------------------
Operation                 LogicalResourceId         ResourceType              R
eplacement
-----------------------------------------------------------------------------
---------------------
+ Add                     ReportLambdaRole          AWS::IAM::Role            N
/A
+ Add                     ReportLambdaURLInvokeA    AWS::Lambda::Permissio    N
/A
                          llowPublicAccess          n
+ Add                     ReportLambdaUrlPublicP    AWS::Lambda::Permissio    N
/A
                          ermissions                n
+ Add                     ReportLambdaUrl           AWS::Lambda::Url          N
/A
+ Add                     ReportLambda              AWS::Lambda::Function     N
/A
+ Add                     ReportsTable              AWS::DynamoDB::Table      N
/A
-----------------------------------------------------------------------------
---------------------


Changeset created successfully. arn:aws:cloudformation:ap-south-1:3825978778
61:changeSet/samcli-deploy1791863251/46bbcfde-dd4f-419f-b7c7-9a0ee558b213
```

The changeset for the urlopen stack: IAM role, Lambda, Function URL and the DynamoDB table.



Real exposure, no login

Opened from a browser with no keys. The function answers anyone. patients: 0 is the count of fake records.

All four stacks are visible in the CloudFormation console. Demo stacks are deleted after the demo.

Safe and broken: one template

infra/template.yaml

```
AuthLambda:
  Condition: IsSafe
ApiAuthorizer:
  Condition: IsSafe
  AuthorizerType: REQUEST
PatientsRoute:
  RouteKey: "ANY /patients"
  AuthorizationType: !If [IsSafe, CUSTOM, NONE]
  AuthorizerId: !If [IsSafe, !Ref ApiAuthorizer, ...]
```

AuthMode=safe

The authorizer Lambda and the API authorizer are created. The route uses CUSTOM auth.

AuthMode=broken

Neither is created. The route has AuthorizationType NONE and no authorizer at all.

One word changes: the guard is simply not there in the broken stack, and every other part still looks fine.

```
AWSTemplateFormatVersion: "2010-09-09"
Transform: AWS::Serverless-2016-10-31
Description: CausalFence toy hospital app

Parameters:
  AuthMode:
    Type: String
    Default: safe
    AllowedValues: [safe, broken]

Conditions:
  IsSafe: !Equals [!Ref AuthMode, safe]
```

powershell

```
sam deploy --stack-name causalfence-safe   --parameter-overrides AuthMode=safe
sam deploy --stack-name causalfence-broken --parameter-overrides AuthMode=broken
```

Open URL and wildcard permission

urlopen-app.yaml

```
FunctionUrlConfig:
  AuthType: NONE
Policies:
  - DynamoDBCrudPolicy:
      TableName: !Ref ReportsTable
```

LAMBDA_URL_OPEN

Anyone on the internet can call this URL. No login.

wildcard-app.yaml

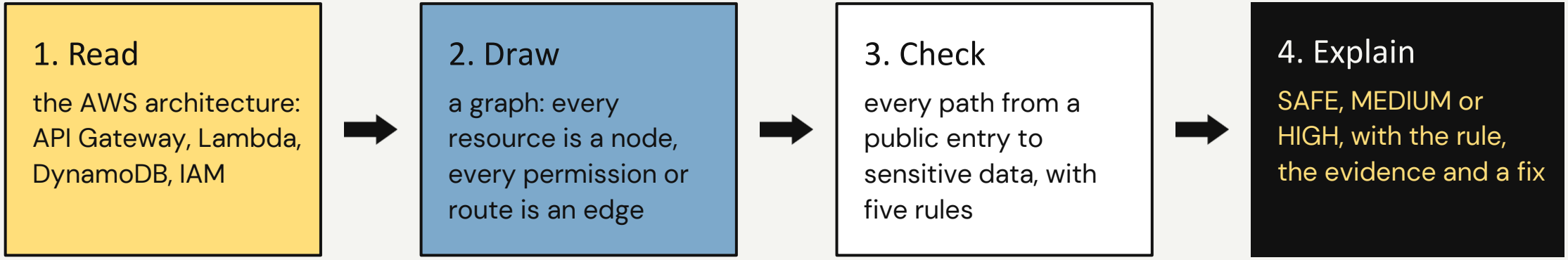
```
FunctionUrlConfig:
  AuthType: AWS_IAM
Policies:
  - Statement:
      - Effect: Allow
        Action: "dynamodb:*"
        Resource: !GetAtt ReportsTable.Arn
```

OVERBROAD_IAM

Allows every DynamoDB action, not just the few the code needs.

Both templates are short. The one risky line is easy to miss, which is why CausalFence reads it for us.

How CausalFence works



BEFORE DEPLOY

iac_parser reads template.yaml in a pull request. No AWS keys. Evidence names the file and line.

AFTER DEPLOY

The Boto3 scanner reads the live AWS stack, so it also catches changes made by hand.

Two ways to build the graph. One engine checks both.

How the verdict is decided

Severity
of the rule: low to critical

Data
PHI counts more than other data

Exposure
public, or public with no auth

Confidence
how sure the evidence is

✓ **SAFE: no findings**

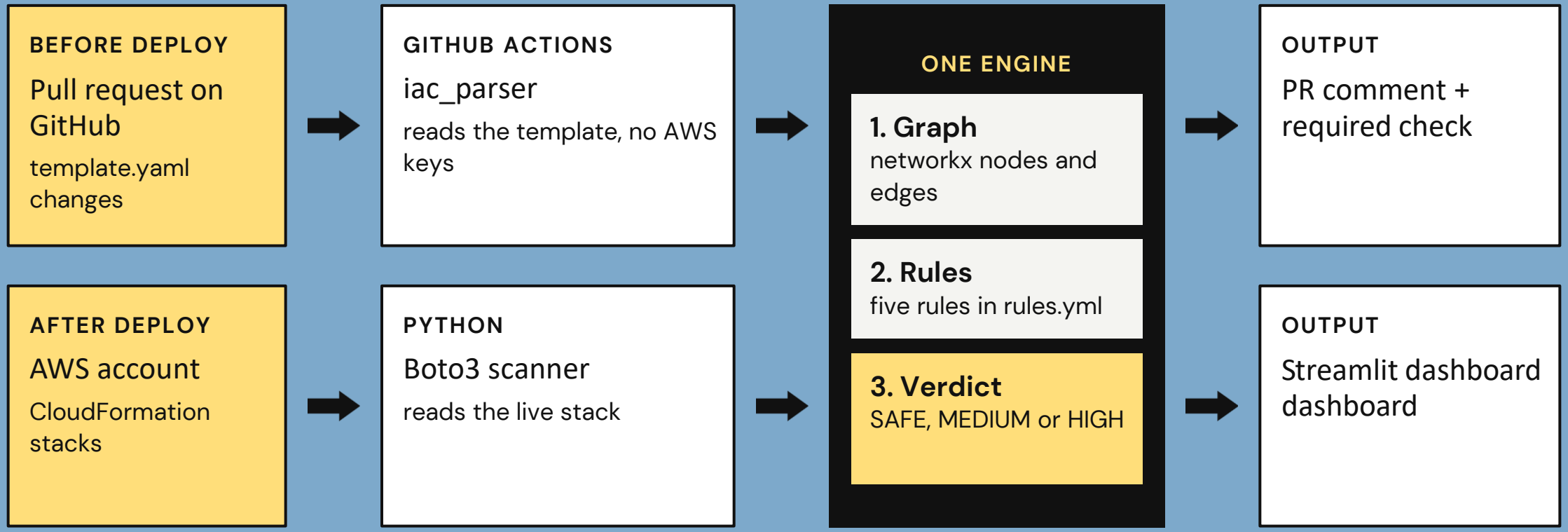
⚠ **MEDIUM: fix soon**

🔒 **HIGH: block it**

```
Result: HIGH RISK - FAIL | Triggered by: AUTH_BYPASS
```

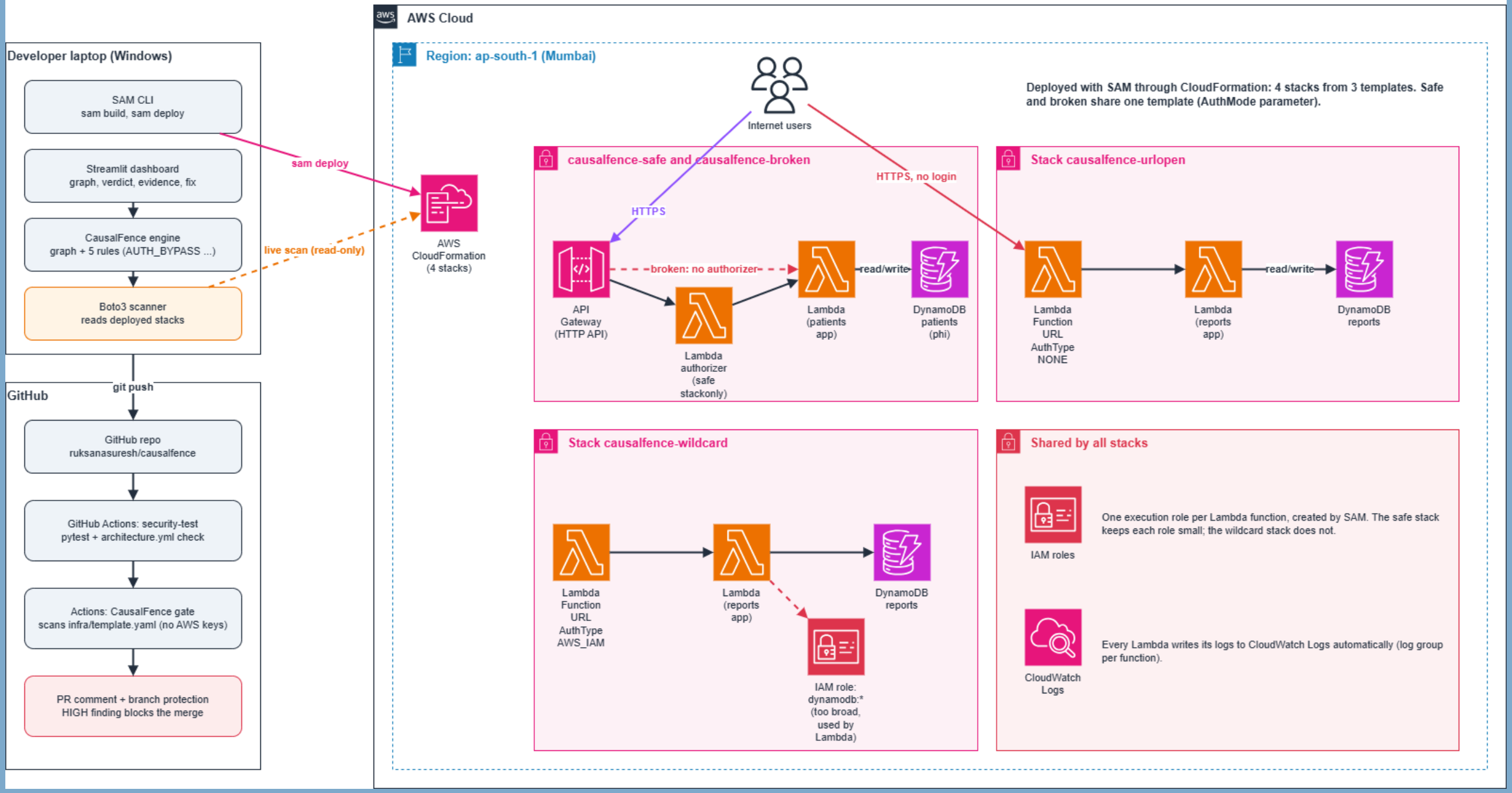
No bare score like 70/100. Every verdict names its rules, shows the evidence, and gives a fix you can paste.

Architecture



Same engine, two ways to build the graph.

CausalFence: AWS Architecture (what is deployed and how it is tested)



Output 1: the dashboard

Pick a deployed stack

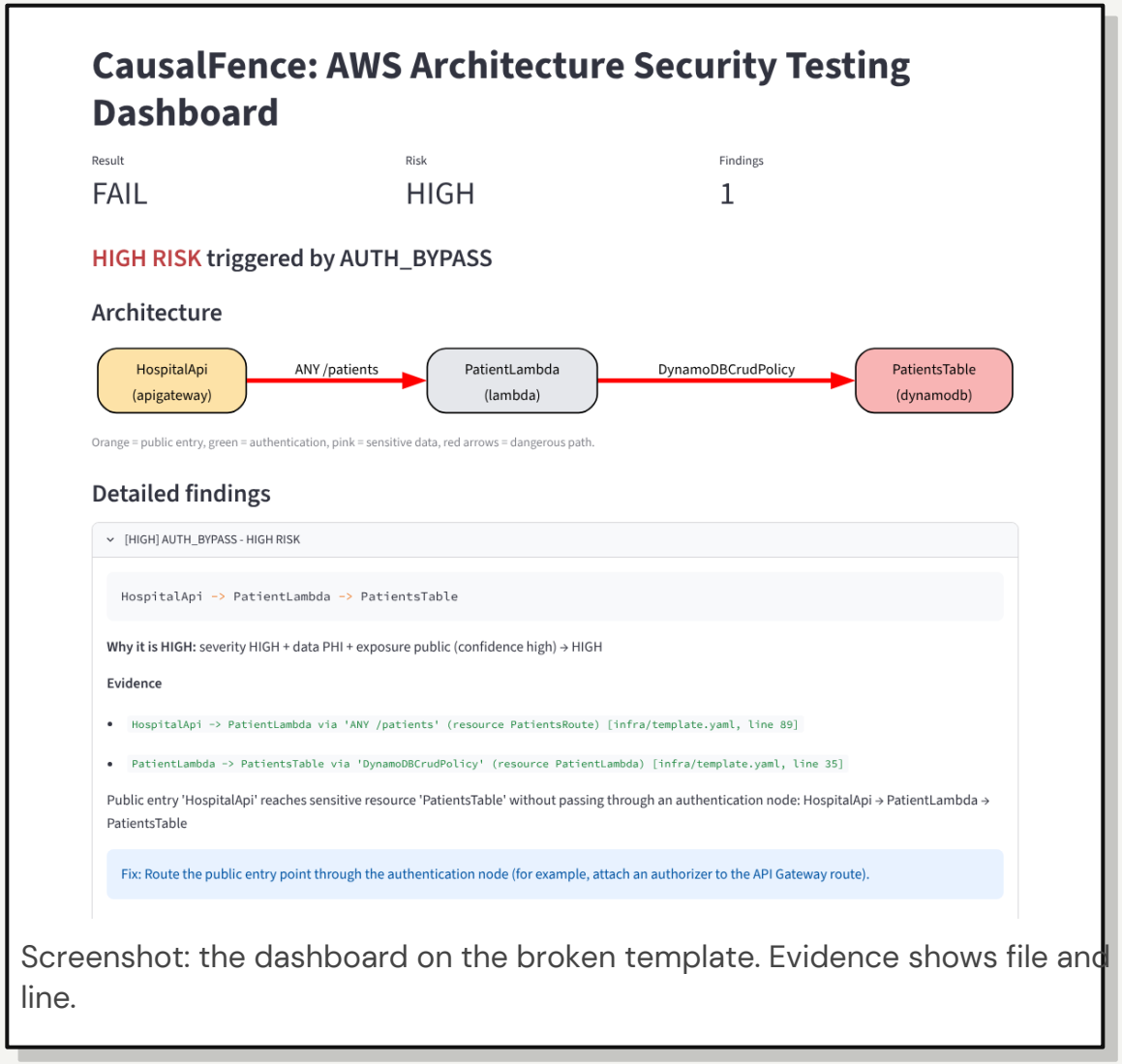
A dropdown lists the stacks in ap-south-1. Click Scan now to read the live account.

See the dangerous path

Red arrows mark the route that skips authentication.

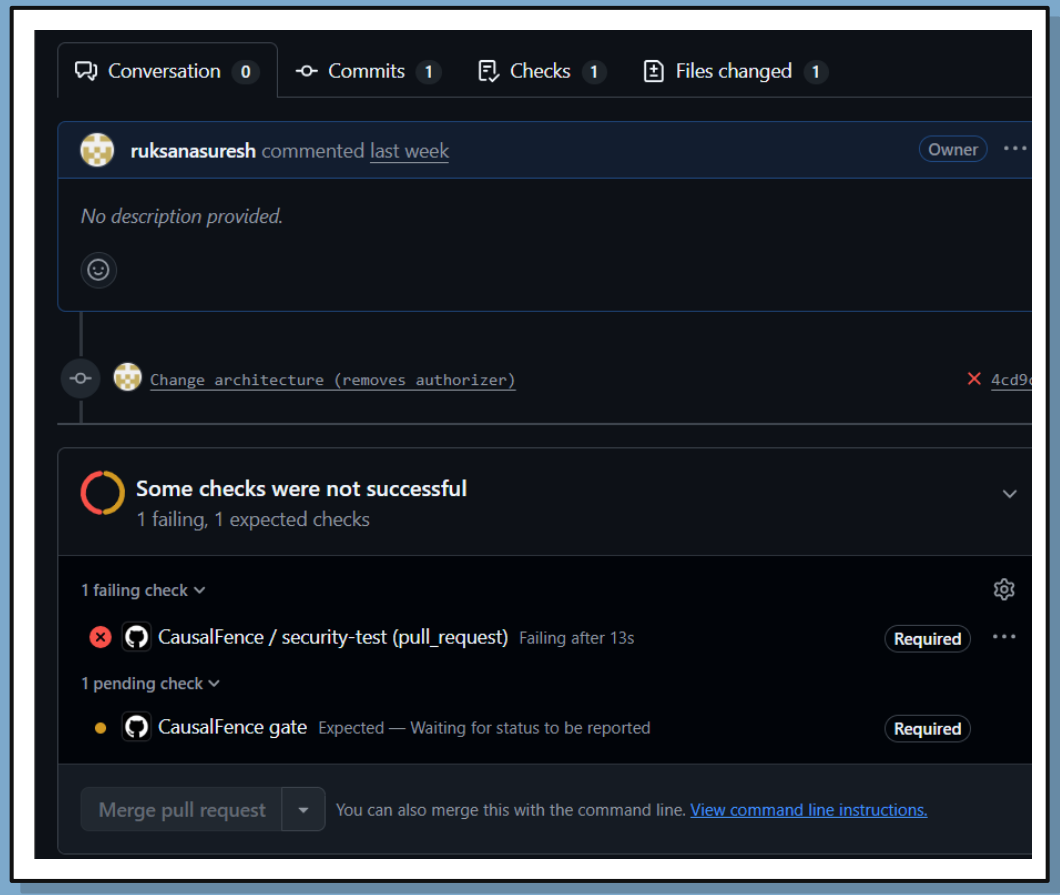
Read why and how to fix it

Verdict, rule, evidence, and a copy-paste fix.



Screenshot: the dashboard on the broken template. Evidence shows file and line.

Output 2: GitHub blocks the merge



PR #1 removes the authorizer

The required check fails and the Merge button stays locked.

PR #2 is safe

Both checks pass and the bot comments: SAFE, no findings.

```
CausalFence security gate: FAIL - merge blocked
HIGH RISK - FAIL | Triggered by: AUTH_BYPASS
```

Comment format when a template is broken: file, line, evidence and fix.

AWS tools we used

API Gateway
HTTP API: the public front door

Lambda
Authorizer, app code, Function URL

DynamoDB
Patient table tagged sensitivity=phi

IAM
Permissions, scoped or wildcard

CloudFormation
Creates and describes the stacks

SAM CLI
sam build and sam deploy

Boto3
Python SDK for the live scanner

AWS CLI
Login check and stack outputs

Also used: GitHub Actions, Streamlit, networkx, PyYAML, pytest.

Why it matters

Catches what single settings miss

It checks every route, not one config.

Blocks risky pull requests early

The check fails before anything deploys.

No AWS keys in CI

It reads a file in the repo, so nothing can leak.

Explains itself

Rule, evidence and a fix, not a bare score.

Catches drift

The live scan shows what is really deployed.

Check the route, not just the setting.

Find the open door before it is built.

github.com/ruksanasuresh/causalfence

Presented by
Ruksana S | 24MISO293

Thank you